

EVA-Ai

Когнитивная нейросетевая система нового поколения

1. Введение и философия системы

1.1 Обзор

EVA — это когнитивная система искусственного интеллекта, построенная на принципах рекурсивного рассуждения, фрактальной памяти и самообучения. В отличие от традиционных языковых моделей, EVA не просто генерирует текст — она анализирует, рассуждает, выявляет противоречия и постоянно совершенствуется.

1.2 Философия системы

EVA построена на фундаментальной идее: **мышление — это не генерация, а процесс**. Каждый ответ проходит через несколько стадий осмысления:

Запрос → Понимание → Анализ → Генерация → Проверка → Синтез → Ответ

Система не стремится выдать первый пришедший в голову ответ. Вместо этого она:

1. **Извлекает факты** — находит релевантные знания в памяти (FractalGraphV2)
2. **Анализирует** — проверяет логическую связность (contradiction detection)
3. **Развивает мысль** — добавляет контекст и детали
4. **Проверяет** — оценивает этичность и достоверность
5. **Синтезирует** — формирует финальный ответ

6. **Запоминает** — сохраняет опыт в граф памяти для самообучения

1.3 Отличие от традиционных LLM

Характеристика	EVA	ChatGPT-4	Claude	Gemini
Архитектура	Three-GGUF Pipeline	Single model	Single model	Single model
Память	Фрактальный граф знаний V2 (SQLite + embeddings)	Контекстное окно	Контекстное окно	Контекстное окно
Обучение	Самодialogи + GraphCurator, автономное	RLHF (человеческая обратная связь)	RLHF	RLHF
Проверка ответов	5+ этапов (этика, противоречия, качество, самодialog)	Один уровень	Один уровень	Один уровень
Fallback	7+ уровней при сбое	Один уровень	Один уровень	Один уровень
Модульность	Встроенные специализированные модули	Внешние API (плагины)	Ограниченно	Ограниченно
Прозрачность	Полная видимость процесса рассуждения	Частичная	Частичная	Частичная
Автономность	Самообучение без внешнего вмешательства	Требует обучения от разработчиков	Требует обучения от разработчиков	Требует обучения от разработчиков
Локальность	Полностью локальная, без облака	Облачная	Облачная	Облачная

1.4 Цели и применение

Целевая аудитория:

- Разработчики, ищущие прозрачную AI-систему
- Исследователи, изучающие когнитивные архитектуры
- Пользователи, ценящие приватность (локальная работа)

Области применения:

- Интеллектуальный ассистент с самообучением

- Аналитическая система с проверкой фактов
 - Образовательная платформа с автономным обучением
-

2. Ключевые инновации

2.1 Трёхуровневый конвейер генерации (Three-GGUF Pipeline)

Система использует три специализированные языковые модели Qwen2.5, каждая из которых выполняет свою функцию в процессе генерации:



Этап 2: Model B – Концептуальное расширение (Qwen2.5-3B)

- Развитие идей из Model A
- Добавление примеров, деталей, контекста
- Строгий контроль повторов (repeat_penalty=2.0)
- Макс. 1024 токенов

Проверка
противоречий
(FractalGraphV2)

Этап 3: Model C – Кодогенератор (Qwen2.5-Coder-1.5B)

- Ленивая загрузка (только при запросе кода)
- Специализированный промпт для кода
- Низкая температура (0.1) для точности
- Макс. 512 токенов

Финальная проверка и синтез

- Этическая проверка
- Проверка противоречий с контекстом
- Сохранение опыта в граф памяти
- save_experience() в FractalGraphV2

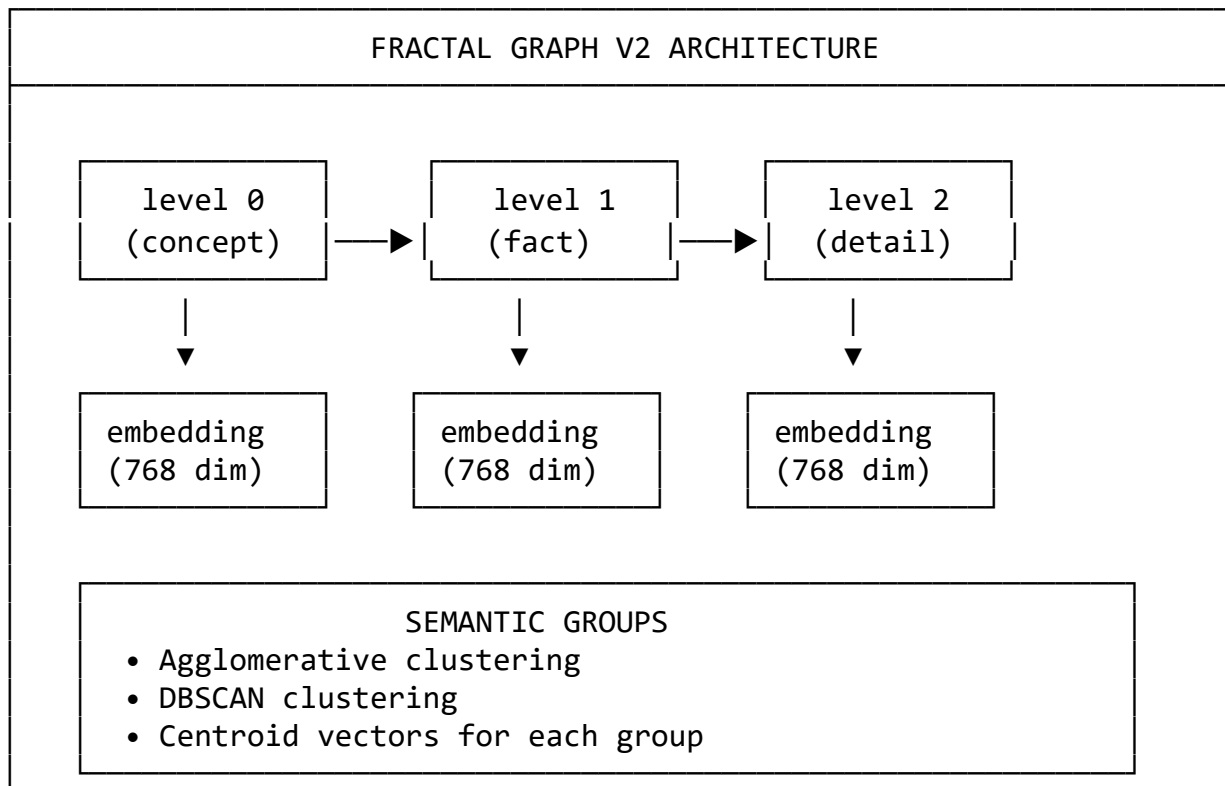


ОТВЕТ ПОЛЬЗОВАТЕЛЮ

- Основной текст ответа
- Раскрываемое меню «Рассуждения» с этапами обработки

2.2 Fractal Graph V2 — Фрактальная память нового поколения

EVA хранит знания в иерархическом графе с векторными эмбедингами:



Компоненты FractalGraphV2:

Компонент	Файл	Назначение
Storage	memory/fractal_graph_v2/storage.py	SQLite база данных, CRUD операции
Embeddings	memory/fractal_graph_v2/embeddings.py	Sentence Transformers (multilingual-e5-base)
Tokenizer	memory/fractal_graph_v2/tokenizer.py	N-gram индексы, кэш контекста
Types	memory/fractal_graph_v2/types.py	FractalNode, SemanticGroup, Edge
GGUF Extractor	memory/fractal_graph_v2/gguf_extractor.py	Извлечение знаний из GGUF моделей

Типы узлов:

Type	Level	Description
concept	0-1	Базовые понятия
fact	1-2	Верифицированные факты
detail	2-3	Детали и уточнения
context	2-3	Контекстуальная информация
attribute	3	Атрибуты и свойства
query	2	Пользовательские запросы
response	2	Ответы системы
learned_knowledge	1	Выученные знания (самодиалог)

Ключевые методы API:

```
# Добавление знаний
add_knowledge(subject, relation, object) # Тройки знаний
add_node(content, node_type, level, confidence) # Узлы

# Поиск
```

```

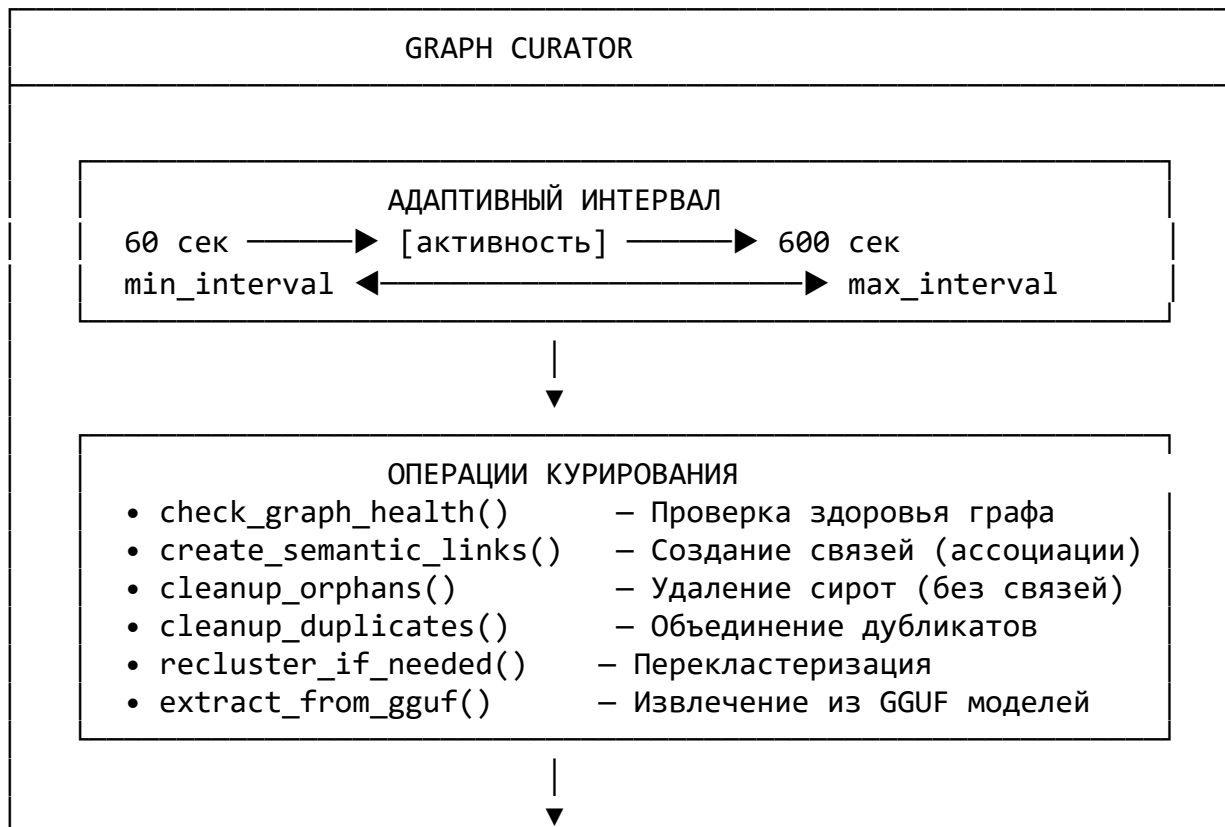
semantic_search(query, top_k)  # Векторный поиск
keyword_search(query, top_k)   # N-gram поиск

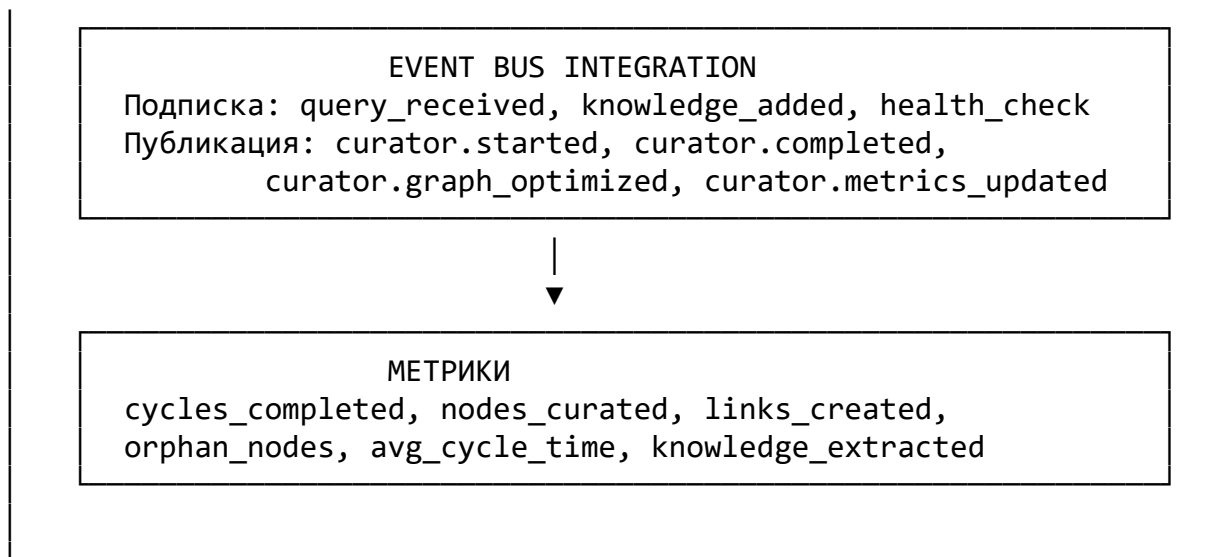
# Обучение и проверка
check_contradiction(content)  # Проверка противоречий
self_dialogue(new_knowledge)   # Верификация через самодialog
save_experience(query, response, model_used, quality_score) # Сохранение опыта
get_context_for_query(query)   # Контекст для генерации

```

2.3 GraphCurator — Куратор графа

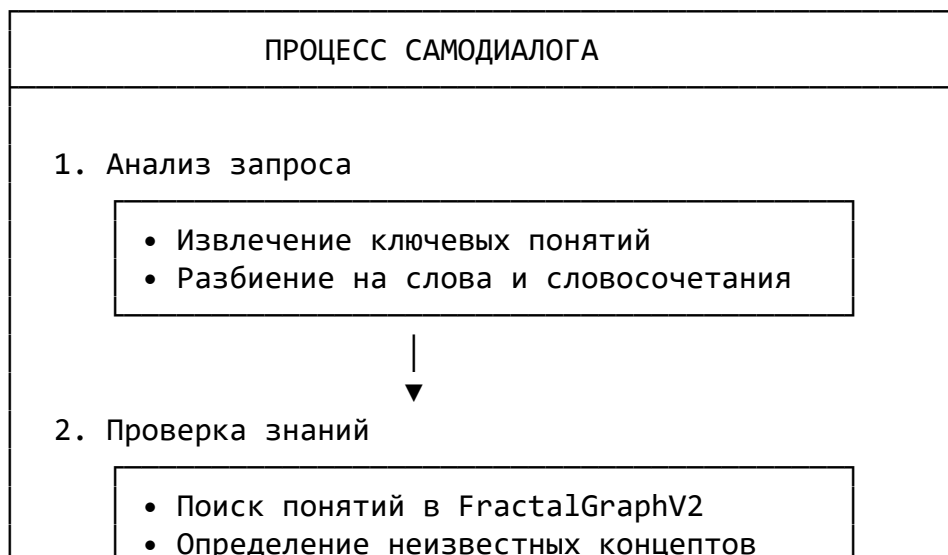
Автоматическая система поддержания порядка в графе знаний:

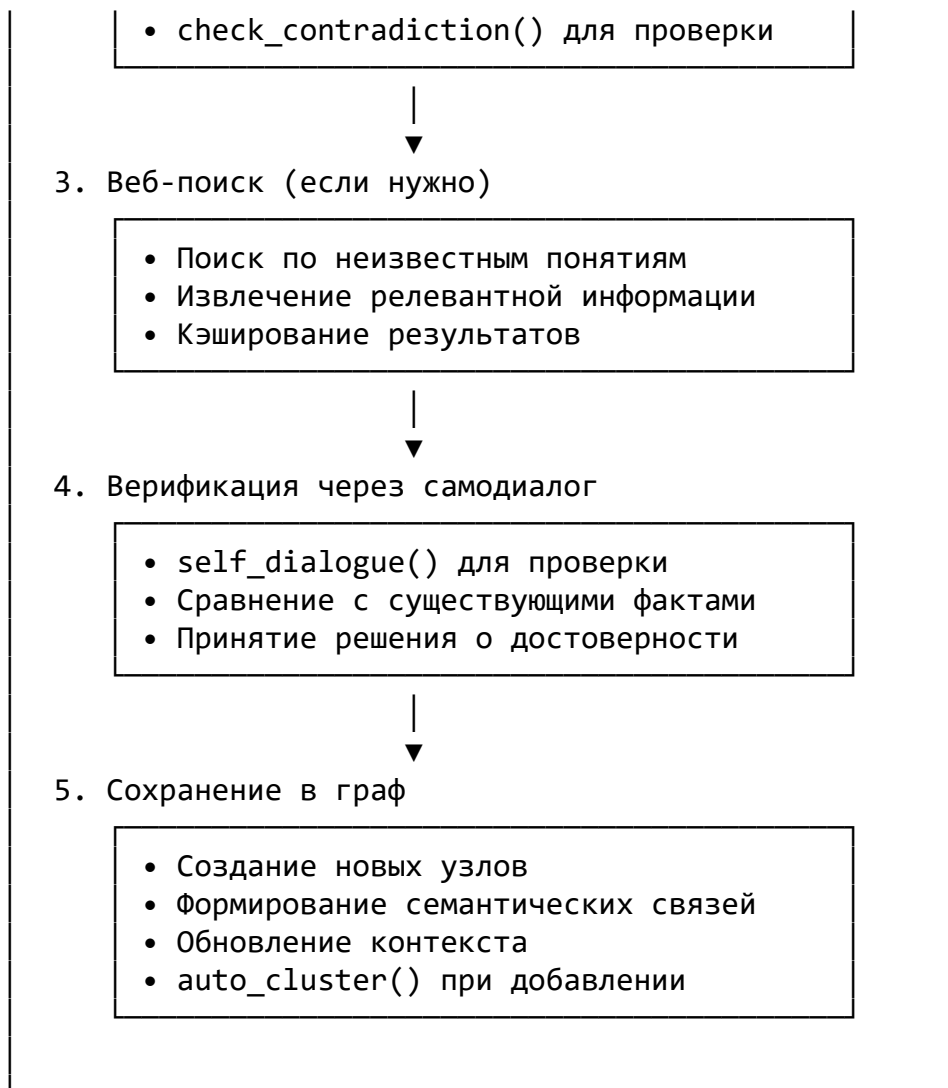




2.4 Система самодialogов

EVA способна самостоятельно изучать новые понятия через процесс самодialogа:





2.5 ConceptMiner — Модуль автономного концептуального вывода

ConceptMiner — проактивный модуль обнаружения семантических лакун во фрактальной памяти:

CONCEPT MINER

СЕМАНТИЧЕСКАЯ ДЕТЕКЦИЯ ЛАКУН

1. Получение события MEMORY_CLUSTERING_COMPLETE
2. Анализ кластеров памяти (centroid distance)
3. Детекция семантических разрывов (semantic_gap > base)
4. Формирование PhantomCandidate – кандидата концепта



ГЕНЕРАЦИЯ ГИПОТЕЗ

1. System prompt: структура определения концепта
2. User prompt: описание лакуны + связанные узлы
3. LLM (temp=0.35) → генерация title, definition, rationale



ВАЛИДАЦИЯ КОНЦЕПТА

1. NLI-когерентность – проверка связности с графом
2. Этическая фильтрация – блокировка опасных гипотез
3. Веб-верификация – подтверждение через внешние источники
4. Lexical entropy адаптация (при entropy < 2.1)



ЖИЗНЕННЫЙ ЦИКЛ КОНЦЕПТА

provisional → confirmed → stable → archived
При CONFIRMED: интеграция с SelfDialogLearning

EventTypes для ConceptMiner:

- `CONCEPT_MINING_START` — начало майнинга
- `CONCEPT_MINING_COMPLETE` — завершение майнинга
- `CONCEPT_CANDIDATE_GENERATED` — новый кандидат
- `CONCEPT_CANDIDATE_CONFIRMED` — подтверждённый концепт
- `MEMORY_CLUSTERING_COMPLETE` — кластеризация памяти завершена

Интеграция:

- GraphCurator публикует `MEMORY_CLUSTERING_COMPLETE` после перекластеризации
- ConceptDialogIntegrator связывает ConceptMiner с SelfDialogLearning
- Конфигурация в `brain_config.json.concept_miner`

2.6 Интеллектуальные модули

Система включает специализированные модули, работающие параллельно с генерацией:

Модуль противоречий

- Сравнивает ответ с FractalGraphV2
- Использует `check_contradiction()` для семантической проверки
- Обнаруживает логические несоответствия через `self_dialogue()`
- Оценивает степень противоречия (расстояние до центра группы)

Модуль этики

- Проверяет запрос и ответ на соответствие этическим нормам
- Оценивает уровень риска
- Блокирует опасный контент до генерации
- Логирует все проверки для аудита

Модуль веб-поиска

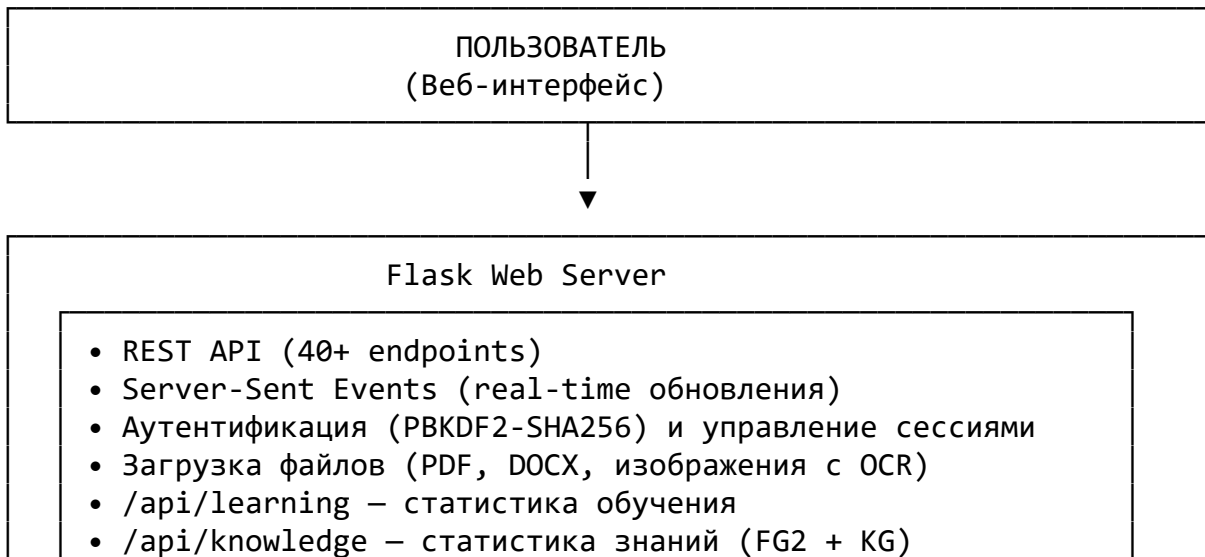
- Автоматически активируется при низкой уверенности
- Пропускает простые запросы (приветствия, благодарности)
- Кэширует результаты для повторного использования
- Интегрирует найденную информацию в контекст

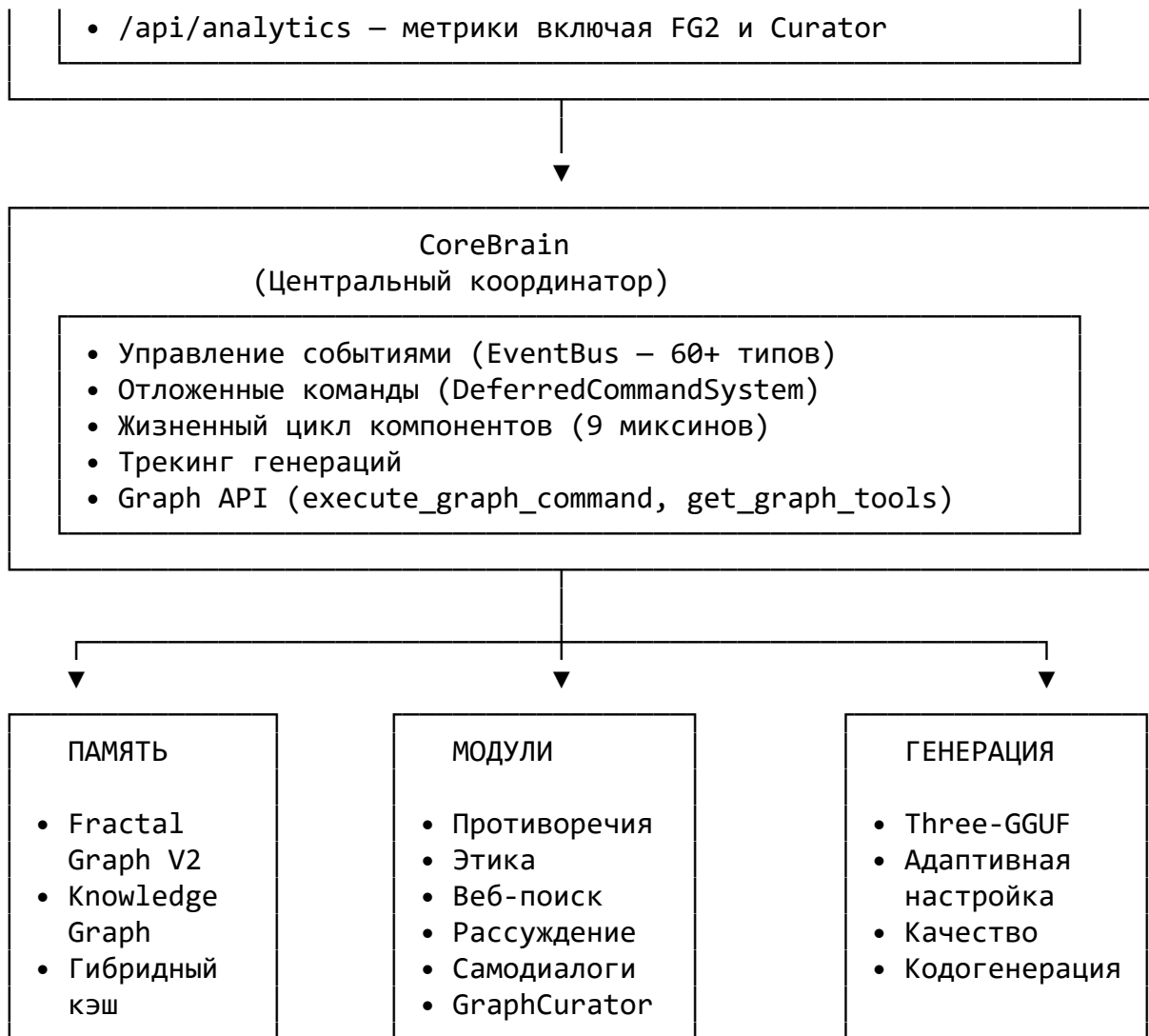
Модуль рассуждения

- Рекурсивный анализ сложных вопросов
 - Многоуровневая проверка гипотез
 - Self-reasoning engine для глубокого анализа
-

3. Архитектура системы

3.1 Высокоуровневая схема





3.2 CoreBrain — центральный координатор

CoreBrain — тонкий координатор, управляющий системой через 9 миксинов:

Миксин	Функция
ConfigMixin	Загрузка конфигурации, маскирование секретов
ComponentMixin	Жизненный цикл компонентов, управление модулями, Graph API
QueryMixin	Многоуровневый fallback chain (7+ уровней), FractalGraphV2 контекст
MonitoringMixin	Health checks, системные метрики
MemoryMixin	Операции с фрактальной памятью
StateMixin	State machine (INITIALIZING → READY → RUNNING → SHUTTING_DOWN)
EventSubscriptionMixin	13 обработчиков событий EventBus
CommandIssuerMixin	20+ типов команд для управления системой
LearningMixin	Интеграция с SelfDialogLearning

Жизненный цикл:

1. `__init__()` — создание EventBus, DeferredCommandSystem
2. `initialize()` — инициализация всех компонентов, переход в READY
3. `start()` — запуск фоновых служб, FractalGraphV2, GraphCurator
4. `stop()` — остановка, переход в SHUTTING_DOWN → OFFLINE
5. `reboot()` — полный цикл `stop` → `reinitialize` → `start`

3.3 EventBus — событийная шина

Централизованная система коммуникации между компонентами:

- **60+ типов событий** — системные, компоненты, обучение, генерация, память, куратор
- **Асинхронная обработка** — события ставятся в очередь, обрабатываются фоновым потоком
- **Автоматическая очистка** — удаление неактивных подписчиков через weak references
- **История событий** — кольцевой буфер на 10,000 событий

Категории событий:

- `system.*` — запуск, остановка, ошибки, готовность
- `component.*` — инициализация, ошибки компонентов

learning.*	– прогресс, завершение, сбои обучения
pipeline.*	– этапы генерации (model_a, model_b, complete)
generation.*	– прогресс, завершение, таймауты
memory.*	– предупреждения, оптимизация
contradiction.*	– обнаружение и разрешение противоречий
ethics.*	– нарушения, предупреждения, проверки
curator.*	– запуск, завершение, оптимизация, извлечение знаний

3.4 DeferredCommandSystem — отложенные команды

Управление фоновыми задачами с приоритетами и защитой от перегрузки:

- **4 очереди приоритетов:** CRITICAL, HIGH, NORMAL, LOW
- **Повторные попытки** с экспоненциальной задержкой
- **Мониторинг здоровья** модулей (интервал 30 секунд)
- **Load shedding** — сброс низкоприоритетных задач при CPU > 80%
- **GraphCurator использует** для отложенных операций курации

3.5 Подсистема памяти

Компонент	Назначение
FractalGraphV2	Основная память (SQLite + embeddings + clustering)
KnowledgeGraph	Legacy граф для совместимости
UnifiedFractalMemory	Устаревший интерфейс (fallback)
HybridTokenCache	Гибридный кэш токенов (RAM + Disk)
EmbeddingCache	SQLite LRU кэш эмбедингов

3.6 Подсистема генерации

Компонент	Назначение
RecursiveModelPipeline	Three-GGUF Pipeline (Model A → B → C)
AdaptiveParameterController	Семантическая детекция заикливания, адаптация

Компонент	Назначение
QualityChecker	Детекция повторений, filler фраз, китайских символов
GenerationTracker	Трекинг жизненного цикла генераций с SSE

3.7 Подсистема обучения

Компонент	Назначение
SelfDialogLearning	Система самодialogов (изучение новых понятий)
DialogLearningMixin	Интеграция с FractalGraphV2
DialogCore	Анализ взаимодействий, сохранение опыта
GraphCurator	Фоновая курация графа знаний

3.8 Веб-интерфейс

SPA (Single Page Application) на чистом JavaScript без фреймворков:

8 представлений:

Представление	Описание
Чат	Основное взаимодействие, markdown, подсветка кода
Память	Визуализация графа знаний, статистика узлов/рёбер
Аналитика	Производительность, метрики CPU/RAM/VRAM/FG2/Curator
Обучение	Возможности обучения, статистика самодialogов
Знания	Поиск по базе знаний (FractalGraphV2 + KG), сущности и отношения
Поиск	Веб-поиск с квотой (1000 запросов/месяц через Tavily API)
Настройки	Конфигурация, экспорт/импорт, статус моделей
Монитор	Real-time события (pipeline, generation, curator, self-dialog)

Особенности:

- Markdown-рендеринг с подсветкой кода и кнопкой копирования

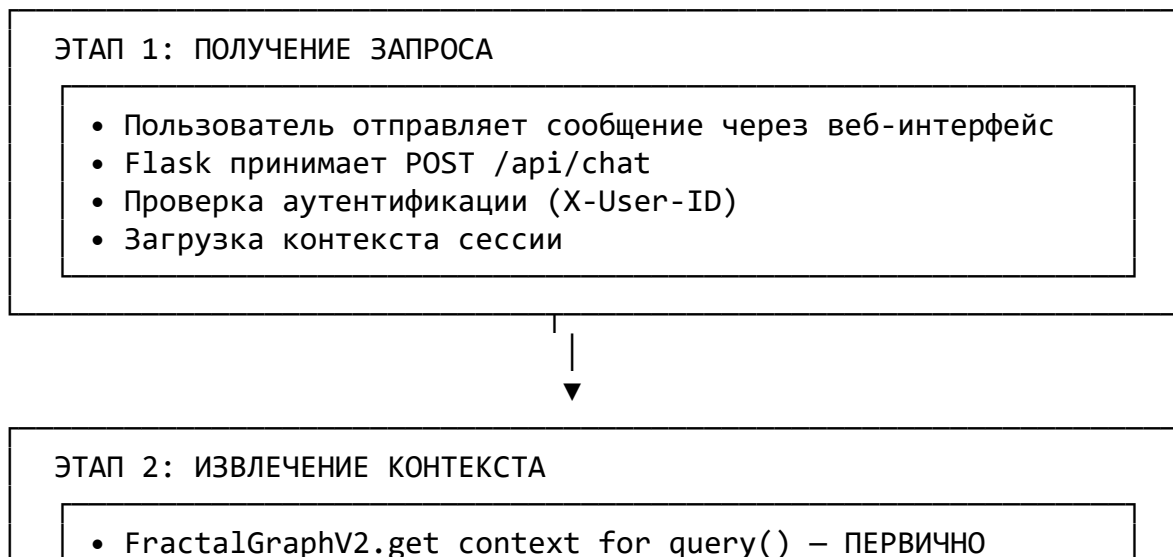
- Раскрываемое меню «Рассуждения» с этапами обработки
- Real-time отображение через SSE (Server-Sent Events)
- Вкладка "Куратор" для мониторинга фоновых операций
- Тёмная тема с CSS custom properties
- Управление сессиями (создание, переименование, удаление)

3.9 Безопасность

- **Аутентификация:** PBKDF2-SHA256, 100,000 итераций, случайная соль
 - **Сессии:** Thread-safe хранение с лимитами (20 context nodes, 50 chat history)
 - **Загрузка файлов:** UUID-именование, поддержка 20+ форматов
 - **API защита:** Проверка токенов, rate limiting
-

4. Поток обработки запроса

4.1 Детальная схема



- `semantic_search()` → извлечение релевантных узлов
- Fallback: `KnowledgeGraph.get_relevant_nodes()`
- Формирование enriched промпта с контекстом



ЭТАП 3: ПРОВЕРКА ЭТИКИ

- Проверка запроса на опасные паттерны
- Оценка уровня риска
- При нарушении → блокировка с объяснением



ЭТАП 4: MODEL A – ЛОГИЧЕСКОЕ ЯДРО

- Генерация краткого фактологического ответа
- Проверка качества (повторения, filler, язык)
- При провале → адаптивная корректировка → retry



ЭТАП 5: MODEL B – КОНЦЕПТУАЛЬНОЕ РАСШИРЕНИЕ

- Развитие идей из Model A
- Добавление примеров и деталей
- Проверка противоречий с `FractalGraphV2`
- `check_contradiction()` → семантическая проверка
- Веб-поиск при низкой уверенности





ЭТАП 6: MODEL C – КОДОГЕНЕРАЦИЯ (при необходимости)

- Ленивая загрузка модели
- Генерация кода с низким temperature
- Проверка синтаксиса



ЭТАП 7: ФИНАЛЬНАЯ ПРОВЕРКА

- Этическая проверка финального ответа
- Проверка противоречий (self_dialogue если нужно)
- Форматирование (markdown, код)



ЭТАП 8: СОХРАНЕНИЕ ОПЫТА

- FractalGraphV2.save_experience()
 - add_node(query, type=query, level=2)
 - add_node(response, type=response, level=2)
 - add_edge(query → response, generated_by)
- При низком rating → SelfDialogLearning.analyze()
- Обновление контекста сессии
- Отправка JSON ответа → SSE streaming → пользователь

4.2 Поток самообучения

USER QUERY



SelfDialogLearning
check_and_create_dialog()



Interest Scorer
is_interesting(query)

True



DialogLearningMixin
_get_fractal_graph()



FractalGraphV2
_learn_expansion()
1. check_contradiction()
2. add_node()
3. auto_cluster()



GraphCurator (background)
create_semantic_links()
cleanup_orphans()

5. Конфигурация системы

5.1 brain_config.json — ключевые секции

```
{
  "fractal_graph_v2": {
    "enabled": true,
    "storage_dir": "eva/memory/fractal_graph_v2/data",
    "embedding_model": "intfloat/multilingual-e5-base",
    "embedding_device": "cuda",
    "embedding_dim": 768
  },
  "graph_curator": {
    "enabled": true,
    "adaptive_interval": true,
    "min_interval": 60,
    "max_interval": 600,
    "check_graph_health": true,
    "cleanup_orphans": true,
    "cleanup_duplicates": true,
    "recluster_threshold": 0.4,
    "extract_from_gguf": true,
    "gguf_models_dir": "eva/mlearning/eva_models"
  },
  "model": {
    "name": "qwen3.5-0.8b",
    "type": "qwen",
    "use_two_model_pipeline": true
  },
  "self_dialog_learning": {
    "enabled": true,
    "auto_execute_enabled": true,
    "auto_learning_interval": 60
  },
  "web_gui": {
```

```
    "host": "127.0.0.1",  
    "port": 5555  
  }  
}
```

6. API Endpoints

6.1 Основные API

Endpoint	Описание
POST /api/chat	Основной чат с генерацией
GET /api/learning	Статистика обучения (SDL + FG2)
GET /api/knowledge	Статистика базы знаний (FG2 + KG)
GET /api/analytics	Метрики системы (CPU, RAM, FG2, Curator)
GET /api/knowledge-graph	Узлы и связи графа
GET /api/events/stream	SSE стрим событий
GET /api/metrics	Ресурсные метрики

6.2 Graph API (через CoreBrain)

```
# Управление через CoreBrain  
brain.execute_graph_command("curator_metrics")  
brain.execute_graph_command("curator_status")  
brain.execute_graph_command("graph_stats")  
brain.execute_graph_command("search <запрос>")  
brain.execute_graph_command("add <субъект> | <отношение> | <объект>")  
  
# Прямой доступ  
brain.fractal_graph_v2.semantic_search("запрос")
```

```
brain.graph_curator.get_metrics()  
brain.graph_curator.force_curation()
```

7. Безопасность и этика

7.1 Этические принципы EVA

Система руководствуется шестью ключевыми принципами:

#	Принцип	Описание
1	Без насилия	Не отвечает на вопросы о насилии, оружии, наркотиках, травле
2	Честность	Если не уверена — говорит об этом, просит уточнение
3	Проверка фактов	Проверяет информацию, исправляет ошибки, не выдаёт догадки за факты
4	Безопасный код	Не генерирует вредоносный или деструктивный код
5	Блокировка рисков	Отклоняет высокорисковые запросы, предупреждает о средних рисках
6	Контроль вывода	Проверяет ответ на этичность перед отправкой

8. Производительность и требования

8.1 Аппаратные требования

Компонент	Минимум	Рекомендуется	Назначение
CPU	6 ядер (AVX2)	8+ ядер	GGUF инференс (llama.cpp)
RAM	16 ГБ	32 ГБ	Модели 3B × 2 = ~4GB, FG2 embeddings ~1GB, система ~6GB
GPU	2 ГБ VRAM (CUDA)	4+ ГБ VRAM	Embedding-модель (e5-base)
SSD	10 ГБ свободно	50+ ГБ SSD	GGUF модели, кэши, FG2 база

8.2 Метрики производительности

Метрика	Значение
Инициализация FG2 (без embedding модели)	~0.5 сек
Инициализация FG2 (с embedding моделью)	~6-8 сек
Семантический поиск (кэшированный)	0.5-0.6 мс
Полная генерация ответа	70-90 мс
Контекст из FG2 (кэшированный)	0.5 мс
Веб-поиск	1-2 секунды
Контекстное окно	до 4096 токенов
GraphCurator цикл	60-600 сек (адаптивно)

9. Технологический стек

Слой	Технология	Версия
Язык	Python	3.10+
LLM Inference	llama.cpp (GGUF)	Актуальная
Языковые модели	Qwen2.5-3B, Qwen2.5-Coder-1.5B	GGUF Q4_K_M
Embedding	Sentence Transformers (multilingual-e5-base)	768-dim
Векторный поиск	NumPy + кэширование	—
База данных	SQLite	—
Веб-сервер	Flask	Актуальная
Интерфейс	Vanilla JavaScript (SPA)	Без фреймворков
Кэширование	Hybrid Token Cache (RAM + Disk)	—
Веб-поиск	Tavily API	1000 запросов/месяц

10. Модульная структура


```
eva/
├── core/                                # Ядро (EventBus, Pipeline, Commands, CoreBrain)
│   ├── brain_init.py                  # Инициализация FG2 + Curator
│   ├── brain_query.py                 # Интеграция FG2 контекста
│   ├── brain_components.py            # Graph API
│   └── init_connections.py            # Dependencies
├── memory/
│   ├── fractal_graph_v2/              # Новая система памяти
│   │   ├── storage.py                 # SQLite хранилище
│   │   ├── embeddings.py              # Sentence Transformers
│   │   ├── tokenizer.py               # N-gram индексы
│   │   ├── types.py                   # FractalNode, SemanticGroup
│   │   ├── gguf_extractor.py          # Извлечение из GGUF
│   │   └── __init__.py                # FractalMemoryGraph API
│   └── unified_fractal_memory.py      # Legacy (fallback)
├── knowledge/
│   └── graph_curator.py               # Модернизированный куратор
├── learning/
│   ├── dialog_core.py                 # SelfDialogLearning + события куратора
│   └── dialog_learning.py             # Интеграция с FG2
├── gui/
│   └── web_gui/
│       ├── server_routes.py           # API /learning, /knowledge
│       ├── static/js/app.js           # Обработка curator events
│       ├── static/css/style.css       # Curator стили
│       └── templates/index.html       # Вкладка Куратор
└── mlearning/                        # ML модели и управление
```

11. Сравнение с аналогичными системами

Характеристика	EVA	ChatGPT-4
Архитектура	3 модели последовательно + FG2	1 монолитная модель
Память	FractalGraphV2 (вектора + кластеры + SQLite)	Контекстное окно
Обучение	Автономное (самодialogи + куратор)	Только разработчиками
Проверка ответов	6 этапов (FG2, противоречия, этика, качество)	Встроенная
Куратор	GraphCurator (фоновый, адаптивный)	Нет
Прозрачность	Полная (SSE монитор, метрики FG2)	Частичная
Приватность	Полностью локальная	Облачная
Стоимость	Бесплатно (локально)	Подписка / API

Контакты

Разработчик: BlackCatSpb
GitHub: <https://github.com/BlackCatSpb/EVA-Ai>
Лицензия: Проприетарная